مهندسی پرامپت، استدلال و تعامل ساختاریافته با مدل‌های زبانی بزرگ (LLMs)

مقدمه و هدف پروژه :

- مدل های زبانی بزرگ ، بر پایه transformer توزیع احتمال شرطی **توکن بعدی** رو بر اساس Context ورودی تعیین میکنند ، از آنجایی که وزن های مدل در زمان **inference** ثابت هستند **کیفیت ، دقت ، قابل اطمینان بودن** خروجی از نحوه مهندسی کردن پرامپت کاربر یا اپراتور نشات میگیره .

در این آزمایش به بررسی کمی و تجربی چهار چالش اصلی در کار با LLMs میپردازیم

1. سوگیری های In-Context Learning : بررسی حساسیت مدل به روش های few-shot نسبت به نا متعادل بودن برچسب مثال ها مثلا (PPPN) و حتی ترتیب قرار گیری مثال ها در پرامپت (برای مثال مدل به مثال های آخر توجه بیشتری میکنه)
2. افزایش ظرفیت استدلال و پایداری خروجی : اغلب مدل به دنبال پاسخ سریع و عجولانه هست ، هدف این است که برای مسائل سخت مدل رو مجبور کنیم اول فکر کنه بعد خروجی تولید کنه ؛ یا مثلا تکنیک Self-Consistency مثلا از مدل تقاضا میکنیم 5 بار از خود سوال را بپرسد و اگر 3 بار پاسخ یکسان بود احتمال زیاد پاسخ درستی تولید کرده (حتی بررسی میکنیم که ارزش داره 5 بار هزینه بدیم بابت دقت بیشتر؟)
3. Grounded RAG : مدل یک حافظه عمومی دارد که دانشش رو از آن میگیرد ، اما تو کاربرد های خاص و دقیق ممکنه دچار توهم و خیال پردازی بشه ، در این مواقع یک سند مرجع به مدل میدهیم و به مدل دستور میدهیم بر این اساس پاسخ تولید کند تا دچار توهم نشود و خیال پردازی نکند.
4. Structured Outputs & Tool Calling: چطور مدل رو وادار کنیم به جای تولید حرف های اضافه دقیقا در همون فرمت مد نظر خروجی تولید بکند (به فرمتی که برنامه ما انتظار داره)

   Structured Outputs : مثلا مدل رو محدود میکنیم تا فقط به صورت JSON خروجی تولید کند

   Tool Calling: مدل می آموزد که چه زمانی خودش پاسخ را تولید کند و چه زمانی از ابزار های بیرونی خودش مثلا (ماشین حساب یا سرویس استعلام قیمت بیت کوین) استفاده کند

آشنایی با ساختار پوشه ها و فایل های پروژه :

```
week 15/
├── src/
│   ├── __init__.py          # پکیج اصلی و مقداردهی اولیه ماژول‌ها
│   └── client.py            # ارتباط یکپارچه و مدیریت کلاینت API (GapGPT)
├── notebooks/
│   ├── 01_few_shot.ipynb                # آزمایش ۱: تحلیل حساسیت و سوگیری‌ها
│   ├── 02_cot_self_consistency.ipynb    # آزمایش ۲: استدلال زنجیره‌ای و رأی‌گیری اکثریت
│   ├── 03_grounded_rag.ipynb            # آزمایش ۳: بازیابی مستند و ارزیابی توهم
│   └── 04_structured_and_tools.ipynb    # آزمایش ۴: خروجی ساختاریافته (Pydantic) و حلقه ابزار (ReAct
└── report/
    ├── assets/              # نمودارها، خروجی‌های ثبت‌شده و تصاویر نتایج
    └── final_report.pdf     # مستند نهایی پژوهش و تحلیل نتایج (گزارش PDF)
```

آزمایش اول few_shot_01 : در این آزمایش zero-shot و few-shot و برعکس کردن ترتیبل مثال ها را بررسی کردیم

داده های تست اولیه برابر این موارد هست :

```
test_sentences = [
    {"id": 1, "text": "Battery lasts all day, best purchase I made this year!", "ground_truth": "Positive"},
    {"id": 2, "text": "Customer service was utterly useless and rude.", "ground_truth": "Negative"},
    {"id": 3, "text": "The cinematography was breathtaking, but the pacing felt a bit dragged.", "ground_truth": "Negative"}, # مرزی
    {"id": 4, "text": "Not bad for the price, does the job well enough.", "ground_truth": "Positive"} # مرزی
]
```

در روش zero-shot صرفا متن instruction به پرامپت های تست اضافه شد

در روش few-shot از دو رویکرد balanced , imbalanced استفاده کردیم روش اول positive:3و سه مثال negative داشتیم ، و در روش imbalanced 5 لیبل positive و 1 لیبل negative داریم ، یک مرتبه هم ترتیب مثال های imbalance رو برعکس کردیم تا تاثیر ترتیب هم بررسی کنیم

مدل مورد استفاده gpt-4o-mini هست و نتایج آزمایش به شرح زیر است :

	id	text	ground_truth	Zero-shot	Balanced	Imbalanced	Imbalanced_Reversed
0	1	Battery lasts all day, best purchase I made th...	Positive	Positive	Positive	Positive	Positive
1	2	Customer service was utterly useless and rude.	Negative	Negative	Negative	Negative	Negative
2	3	The cinematography was breathtaking, but the p...	Negative	Negative	Negative	Negative	Negative
3	4	Not bad for the price, does the job well enough.	Positive	Positive	Positive	Positive	Positive

همان طور که مشاهده میکنید به علت قدرت بالای مدل و سادگی تست ها عملکرد کاملا بی نقص بود یعنی accuracy 100% اما همان طور که حدس زدید ، من تا زمانی که مدل رو به زانو در نیارم دست نمی کشم پس منتظر باشید چون قراره داده های تست رو فوق العاده سخت کنم با اضافه کردن یک لیبل ناشناخته (Neutral) ، و جملاتی که مبهم هستن و تشخیص لیبل اونا کار ساده ای نیست

داده های تست مرگبار عبارتند از:

```
test_sentences_hard_test = [
    {    "id": 1,
        "text": "Oh wonderful, another software update that broke every single feature I actually use.",
        "ground_truth": "Negative"},

    {   "id": 2,
        "text": "The package arrived at 3 PM on Tuesday and matched the dimensions on the box.",
        "ground_truth": "Neutral"},

    {   "id": 3,
        "text": "The design is sleek and premium, though it frequently overheats and crashes under normal load.",
        "ground_truth": "Negative"},

    {   "id": 4,
        "text": "It simply exists. It does what it is supposed to do, nothing more and nothing less.",
        "ground_truth": "Neutral"}
]
```

نتایج تست روی این داده ها به این صورت هست:

	id	text	ground_truth	Zero-shot	Balanced	Imbalanced	Imbalanced_Reversed
0	1	Oh wonderful, another software update that bro...	Negative	Positive	Positive	Positive	Positive
1	2	The package arrived at 3 PM on Tuesday and mat...	Neutral	Negative	Negative	Negative	Negative
2	3	The design is sleek and premium, though it fre...	Negative	Negative	Negative	Negative	Negative
3	4	It simply exists. It does what it is supposed ...	Neutral	Positive	Positive	Positive	Positive

بله مدل رو به زانو در آوردیم به طوری که accuracy فقط 25٪ درصد شد البته علت آن به وضوح استفاده از یک لیبل ناشناخته و یک مورد که کاربر به کنایه داشته صحبت میکرده (id:1)

در واقع در این مرحله تونستیم مشاهده کنیم که تعریف نکردن لیبل Neutral و کنایه آمیز بودن و پیچیده بودن نظر کاربر تونست عملکرد مدل رو دچار چالش کنه .

حالا بهتره یه مقدار به مباحث علمی و تعریف روش هایی که تست کردیم بپردازیم :

مبانی تئوری و مکانیزم سوگیری‌ها درIn-Context Learning

یادگیری درون‌متنی (In-Context Learning) به این معناست که مدل بدون تغییر در وزن‌های اصلی، صرفاً با تکیه بر توکن‌هایی که در ورودی (Context) دریافت می‌کند، خروجی را پیش‌بینی می‌کند. در این فرآیند، چیدمان ورودی به اندازه دانش قبلی مدل اهمیت دارد و متغیرهای زیر مستقیماً بر رفتار مدل اثر می‌گذارند:

۱ .دستورالعمل (Instruction) و یادگیری بدون مثال(Zero-shot)

دستورالعمل، تعریف صریحی از وظیفه (Task) است که قالب خروجی و دسته‌بندی‌های مجاز را برای مدل مشخص می‌کند. در رویکردZero-shot ، مدل هیچ نمونه‌ای برای یادگیری ندارد و صرفاً به دانش پیشین خود تکیه می‌کند. اگر متن ورودی واضح باشد، این روش عملکرد خوبی دارد؛ اما وقتی ورودی مبهم است یا در توزیع دانش مدل نیست، چون نمونه‌ای برای الگوبرداری ندارد، مدل فاقد چارچوب مقایسه‌ای برای کالیبره کردن پاسخ خود است و دچار خطا می‌شود.

۲ .یادگیری با چند مثال(Few-shot)

ارائه چند نمونه ورودی-خروجی، فضای تصمیم‌گیری را برای مدل محدودتر و دقیق‌تر می‌کند، اما این کار سوگیری‌های رفتاری خاصی ایجاد می‌کند:

رویکرد متوازن:(Balanced Few-shot)

وقتی تعداد مثال‌های مثبت و منفی برابر باشد (مثلاً ۳ به ۳)، مدل به سمت هیچ‌کدام متمایل نمی‌شود و توزیع احتمالِ پیشین برای هر دو کلاس تقریباً برابر باقی می‌ماند. این حالت استانداردترین وضعیت برای سنجش دقتِ خالص مدل است.

سوگیری برچسب اکثریت:(Majority Label Bias / Imbalanced)

وقتی تعداد مثال‌های یک کلاس (مثلاً مثبت) به‌مراتب بیشتر از کلاس دیگر باشد، مدل به‌اشتباه تصور می‌کند توزیع کلی داده‌ها همین است. در نتیجه اگر یک جمله مبهم یا خنثی دریافت کند، چون احتمال برچسبِ اکثریت را در کانتکست بیشتر دیده، تمایل دارد به همان پاسخ دهد. این رفتار نشان‌دهنده غلبه‌ی الگویِ درون‌متنی بر دانشِ معناییِ مدل است.

سوگیری تقدم و تأخر:(Recency Bias / Imbalanced Reversed)

مکانیزمِ توجه (Attention) در مدل‌های زبانی، به ترتیبی که مثال‌ها آورده می‌شوند حساس است. مدل‌ها معمولاً به مثال‌هایی که به جمله هدف نزدیک‌ترند (مثال‌های آخر)، وزن بیشتری می‌دهند. با معکوس کردن ترتیب در حالت نامتوازن، بررسی کردیم که آیا مدل می‌تواند با دیدن مثالِ مخالف در انتهای کانتکست، بایاسِ قبلی خود را بشکند یا همچنان تحت‌تأثیرِ ترتیبِ مثال‌ها باقی

می‌ماند. این آزمایش نشان داد که در مواجهه با داده‌های پیچیده، وزنِ مثال‌های انتهایی کانتکست می‌تواند به‌سادگی نتیجه را تغییر دهد.

آزمایش 02 Cot and self-consistency :

در این آزمایش سه رویکرد رو تست خواهیم کرد ، رویکرد اول zero-shot هست که هیچ کدوم از دو روش cot , self-consistency رو نداریم و مدل صرفا یه پرامپت ساده دریافت میکن برای حل مسئله ای که حتی ذهن انسان رو درگیر میکند و بدون کاغذ قابل حل نیست ، قطعا مدل با عجله کردن و دقت و بررسی ناکافی پاسخ های اشتباهی تولید خواهد کرد

قبل از هر چیزی بیاید نگاهی به داده ای تست بندازیم و پیشنهاد میکنم که سرعت و دقت خودتون در حل این سوالات رو محک بزنید :

```
questions = [
    {
        "id": 1,
        "question": "A bakery makes 48 cupcakes in the morning. They sell 3/4 of them by noon. In the afternoon, they bake another 30 cupcakes and sell
         1/3 of the total cupcakes they currently have. How many cupcakes are left at the end of the day?",
        "ground_truth": 28
    },
    {
        "id": 2,
        "question": "A train leaves Town A at 2:00 PM traveling at 60 mph towards Town B. Another train leaves Town B at 3:00 PM traveling at 80 mph
         towards Town A. The distance between Town A and Town B is 270 miles. At what time (in PM, just write the hour as integer, e.g. 4) will they
         meet?",
        "ground_truth": 4
    },
    {
        "id": 3,
        "question": "Amir has 5 shirts, 3 pairs of pants, and 2 pairs of shoes. However, 1 of the shirts does not match with 1 of the pants (this
         specific combination cannot be worn). How many total unique complete outfits (1 shirt, 1 pants, 1 shoes) can he make?",
        "ground_truth": 28
    },
    {
        "id": 4,
        "question": "A snail is at the bottom of a 20-foot well. Each day it climbs up 5 feet, but each night it slides down 2 feet. On which day will
         the snail finally reach the top of the well?",
        "ground_truth": 6
    }
```

یک نکته مهم ، دراین آزمایش ما از یک تابع استفاده کردیم تا جواب مدل رو به صورت یک عدد واحد برگردونه تا بتونیم اکیوریسی رو محاسبه کنیم ، که کارکرد این تابع به این صورت هست

```python
def extract_final_number(text: str):
    """extract answer"""
    matches = re.findall(r"[-+]?\d*\.?\d+", text.replace(",", ""))
    if matches:
        val = matches[-1]
        return int(float(val)) if float(val).is_integer() else float(val)
    return None
```

در ادامه بهتره به جداولی توجه کنیم که عملکرد مدل ، پاسخ صحیح ، میزان توکن مصرفی و متریک های دیگه رو به ما اعلام میکنن تا ببینیم هر رویکرد چه طور توانسته روی کار ما تاثیر بگذارد :

=== Summary Comparison ===

	Method	Accuracy (%)	Avg Tokens
0	Direct	0.0	90.25
1	Zero-shot CoT	100.0	449.50
2	Self-Consistency (k=3)	100.0	1454.00

=== Detailed Predictions ===

	id	ground_truth	Direct_Pred	CoT_Pred	SC_Pred
0	1	28	26	28	28
1	2	4	5	4	4
2	3	28	30	28	28
3	4	6	18	6	6

تفسیر عملکرد : بدترین عملکرد مربوط به direct-prompt(zero) بوده ، هیچ نمونه تستی رو درست حل نکرده است ، و کمترین توکن مصرفی متعلق به این روش است پس هزینه کمتر ولی دقت 0 دارد پس عملا بی کاربرد بوده ؛ در تست بعدی هنوز zero-shot هستیم ولی با این تفاوت که به مدل تاکید میکنیم عجله نکند و مرحله به مرحله فکر کند با این پرامپت

```
"Let's think step by step to find the correct solution. "
"At the very end of your response, write the final numerical answer clearly
on a new line in this format: 'Final Answer: <number>'."
```

که باعث یک رشد شگفت آور در دقت شده و دقت به 100 درصد رسیده میزان توکن مصرفی 450 توکن هست به طور میانگین برای هر تست و تا اینجا بهترین عملکرد مشاهده شده هست

در آزمایش بعدی از تکنیک self-consistency استفاده کردیم و از مدل سه مرتبه سوال پرسیدیم و رای اکثریت رو به عنوان مرجع در نظر گرفتیم و temperature رو به 0.7 رسوندیم تا پاسخ تکراری تولید نکنه

توکن مصرفی افزایش چشم گیری داشت 1450 توکن برای هر تست ولی دقت یکسان بود
در نهایت با وجود محدود بودن داده های تست میشه نتیجه گرفت که بهینه ترین روش با اختلاف **Chain of Thought** بود.

مبانی تئوری استدلال گام‌به‌گام (Chain-of-Thought) و خودسازگاری(Self-Consistency)

مدل‌های زبانی بزرگ ذهن یا حافظه کاری پنهان به معنای سنتی ندارند؛ محاسبات آن‌ها در هر گام صرفاً به توکن‌های تولیدشده تا آن لحظه وابسته است. در مسائل استدلالی، اگر از مدل خواسته شود بلافاصله جواب نهایی را بگوید، مجبور می‌شود کل مسیر

منطقی را در یک فوروارد پس (Forward Pass) خلاصه کند که این موضوع ریشه اصلی خطاهای محاسباتی است. در این آزمایش، سه رویکرد برای حل این مشکل مقایسه شدند:

۱ .رویکرد مستقیم(Direct Prompting)

در این حالت، مدل ملزم می‌شود بدون مقدمه، تنها پاسخ عددی یا نهایی را تولید کند. از نظر تئوری، مدل مجبور است متغیرها، گام‌های میانی و عملیات چندمرحله‌ای را بدون تخصیص فضای پردازشی حل کند. این روش اگرچه از نظر تعداد توکن بسیار کم‌هزینه و سریع است، اما در مسائل دارای چند زیرمسئله (Multi-step) به‌شدت مستعد خطای حدس زودهنگام و محاسبات اشتباه است.

۲ .زنجیره استدلال(Chain-of-Thought & CoT) :

با افزودن عباراتی نظیر «گام‌به‌گام فکر کن»، مدل را وادار می‌کنیم توکن‌های میانیِ تفکر را روی پنجره کانتکست بنویسد. اهمیت تئوریک این روش در دو عامل نهفته است:

تخصیص زمان محاسباتی بیشتر: مدل به ازای هر توکن جدید یک مرحله استنتاج انجام می‌دهد؛ بنابراین نوشتن مراحل استدلال عملاً به مدل اجازه می‌دهد توان پردازشی بیشتری صرف کند.

استفاده از حافظه کاری: هر نتیجه میانی که تولید می‌شود، به ورودیِ گام‌های بعدی اضافه می‌شود. به این ترتیب مدل در هر مرحله فقط یک عملیات ساده را با تکیه بر اطلاعات نوشته‌شده در گام قبل حل می‌کند و فشار شناختی از روی یک توکن برداشته می‌شود. هزینه این روش، افزایش مصرف توکن و تاخیر در پاسخ است.

۳ .روش خودسازگاری(Self-Consistency)

رویکرد قبلی استدلال را فعال می‌کند، اما هنوز بر پایه دمای صفر و یک مسیر خطی واحد است؛ اگر مدل در یکی از مراحل میانی دچار لغزش شود، کل پاسخ غلط از آب درمی‌آید. رویکرد خودسازگاری بر پایه این فرض آماری بنا شده است که «یک مسئله استدلالی پیچیده معمولاً راه‌های متفاوتی برای تحلیل دارد، اما پاسخ درست یکسان است؛ در حالی که مسیرهای غلط به جواب‌های متفاوتی منجر می‌شوند.»

مکانیزم عملکرد: دما را کمی افزایش می‌دهیم تا مدل مسیرهای مختلفی برای حل مسئله پیشنهاد دهد (تنوع در استدلال). سپس در پایان، روی پاسخ‌های نهایی رای‌گیری اکثریت انجام می‌شود.

تعادل دقت و هزینه: این تکنیک، خطاهای تصادفی در محاسبات را تا حد زیادی حذف می‌کند و پایداری سیستم را به حداکثر می‌رساند، اما به دلیل اجرای چندباره، هزینه محاسباتی و توکن‌های مصرفی را مستقیماً چند برابر افزایش می‌دهد.

آزمایش RAG_03 ساده:

در این آزمایش به بررسی RAG یا بازیابی اطلاعات تقویت شده پرداختیم ، در این روش به مدل یک پایگاه داده میدهیم تا پاسخ مراجعین را با دانش خود ندهد و با اطلاعات موجود در این پایگاه داده پاسخ بدهد ، مسئله این است که ، ما باید طوری مدل را مورد پرسش قرار دهیم که وقتی اطلاعاتی در پایگاه داده وجود ندارد به طور واضح بگوید I don’t know یا هر پیغام مدنظر دیگر، نه اینکه شروع به توهم زدن و خیال پردازی کند ، مسئله مهم بعدی این است که پایگاه داده پر از اسناد مختلف است ! اگر سند مد نظر ما در اول این پایگاه داده باشد ! در اواسط آن باشد ! یا در پایان آن باشد میتواند روی خروجی تاثیر گزار باشد پس این مسئله را نیز بررسی کردیم و در کل به دو آزمایش روی این قضیه پرداختیم ،تغییر محل golden document و تعریف یک سوال خارج از دیتابیس مرجع .

حال که در رابطه با آزمایش توضیحات کلی دادم بهتر است که با دیتابیس مرجع آشنا بشویم :

```
docs = {
    "doc_gold": "The secret ingredient in the X-102 energy drink is 'Blue Moss Extract' harvested
from the Caspian Sea.",
    "distractor_1": "The global production of lithium is expected to double by 2030 due to EV
demand.",
    "distractor_2": "Mount Everest is officially 8,848.86 meters tall according to recent
surveys.",
    "distractor_3": "The first decentralized digital currency was created in 2009 by an unknown
person."
}

question = "What is the secret ingredient in the X-102 energy drink?"
```

همان طور که مشاهده میکنید تنها یکی از اسناد پاسخ درست سوال ما هست و بقیه اسناد به سوال تست بی ربط هستند

در ادامه مدل را در سه سناریو به تست گزاشتیم :

Golden document → beginning

Golden document →Middle

Golden document →End

جدول خروجی به این صورت شد :

	Position	Response	Is_Correct
0	Beginning	The secret ingredient in the X-102 energy drin...	True
1	Middle	The secret ingredient in the X-102 energy drin...	True
2	End	The secret ingredient in the X-102 energy drin...	True

مدل قدرتمند در هر سه آزمون سر بلند بیرون آمد ولی باید توجه داشت که معمولا محل قرار گیری Golden Doc در دقت مدل تاثیر گزار است. و به علت طول بسیار کم داکیومنت های ما (فقط 4 سند) این پدیده آشکار نشده ! و معمولا در تعداد بیشتری از اسناد این پدیده خود را نشان می دهد .

در آزمایش بعدی از مدل سوالی کردیم که جواب آن در دیتابیس وجود ندارد و انتظار داریم I don’t know پاسخ مدل باشد

```
"What is the capital of the Martian Colony established in 2045?"
```

Model Response: I don't know.

و این یک موفقیت است که مدل بدون هیچ توضیح اضافه ای و حتی بدون خیال پردازی پاسخ درست داده

در نهایت این شما و این نتایج درخشان آزمایش 03 :

```
=== RAG Experiment Results ===
Position: Beginning -> ✅ Success
Position: Middle -> ✅ Success
Position: End -> ✅ Success

=== Hallucination Analysis ===
Hallucination Test Result: ✅ Grounded (Refused hallucination)
Response: I don't know.
```

آزمایش structured_&_tools_04 : همان طور که حتما میدانید ، مدل های زبانی تنها کاری که میتوانند انجام دهند تولید متن است ، لذا اگر قرار باشد از ابزاری استفاده کنند باید یک درخواست شفاف و دقیق در متن خروجی خود تولید کنند ، انجام این عمل چالش هایی دارد که به انها در ادامه اشاره خواهم کرد :

1. خروجی غیر قطعی و بدون ساختار : مدل ها متون آزاد تولید میکنند در حالی که پایپ لاین های نرم افزاری نیاز مند آبجکت های ساخت یافته ، معتبر (از نظر ترتیب ، نوع ، فرمت خروجی) ، و مقید به قرار داد های مشخص هستند (schema)
2. فقدان قطعیت محاسباتی و عدم دسترسی به داده های بلادرنگ : مدل های زبانی به دانش ایستا که در زمان آموزش به آنها داده شده محدود هستند و در مسائلی مثل فرمول های قطعی مستعد خطا هستند چون با تولید یک توکن اشتباه کل مسیر نابود میشود

هدف ما در این آزمایش : ایجاد فضایی که LLM بتواند از این ابزار ها استفاده کند به نحوی که تکرار پذیر ، دقیق و قابل اعتماد باشد .

Data Schema & Validation Contract : هدف و فلسفه طراحی این بخش ← برای اتصال خروجی مدل به لایه های بعدی باید از نوع داده ها مطمئن بشویم ، پس در این گام با کتابخانه Pydantic قرار داد نوع و شکل داده بین مدل و ابزار ها رو تدوین میکنیم

: SentimentAnalysis

```
class SentimentAnalysis(BaseModel):
    reasoning: str
    label: Literal["positive", "negative", "neutral"]
```

مدل استاندارد اعتبار سنجی خروجی ، که در این روش اول مدل استدلال از نوع string تولید میکند سپس یک لیبل از بین این سه گزینه ! ارزش این کار به این است که مدل های transformer از نوع decoder-only میتوانند فقط به توکن های گذشته تولید کنند ، اگر به انها اجازه بدهیم اول استدلال کنند بعد لیبل انتخاب کنند ، به انها کمک میکنیم با attention کردن به توکن های قبلی خود لیبل را انتخاب کنند

: SentimentAnalysis Reversed

```
class SentimentAnalysisReversed(BaseModel):
    label: Literal["positive", "negative", "neutral"]
    reasoning: str
```

این بار دقیقا برعکس روش قبل عمل کردیم و مدل رو دچار چالش خواهیم کرد و ممکن است بدون داشتن reasoning خود ، نتواند لیبل خوبی تولید کند .

```
reviews = [
    "The battery life is stellar, lasting two full days easily.",
    "Customer service was utterly unhelpful and rude.",
    "The camera is decent, but the software has frequent glitches.",
    "Not worth the ticket, though the visual effects were neat.",
    "It arrived on time."
]
```

ترکیب مجموعه داده ارزیابی(reviews)

دادگان آزمون متشکل از ۵ نمونه هدفمند جهت ارزیابی در شرایط مرزی است:

نمونه‌های شفاف :جملات ۱ و ۲ با لیبل صریح.(Positive/Negative)

نمونه‌های متناقض و مرزی :(Mixed/Ambiguous) جملات ۳ و ۴ که شامل نکات مثبت و منفی توأم بوده و نیازمند تفکیک وزن احساسی هستند.

نمونه خنثی (Objective/Neutral): جمله ۵ فاقد بار احساسی مستقیم برای ارزیابی کلاس neutral.

در سلول بعدی نوت بوک به سه رویکرد خروجی گرفتن از مدل میپردازیم :

1. صرفا instruction : ریسک نوع داده و رفتار تصادفی مدل و آلودگی به مارک داوون و...
2. خروجی json تضمین شده : ولی ترتیب و درستی کلید ها تضمین شده نیست
3. تعیین دقیق فرمت و schemas : اینبار همه چیز کاملا تضمین شده و مناسب کال کردن ابزار هاست

:1

```
system_instruction = (
    "Analyze the sentiment of the review. "
    "Allowed labels: positive, negative, neutral."
)
```

صرفا با این دستور از مدل تقاضا میکنیم ما رو راضی کند و نتیجه درست تولید کند که اغلب این کار را نمی کند

:2

```
# 2. JSON Mode
res2 = client.chat.completions.create(
    model="gpt-4o-mini",
    response_format={"type": "json_object"},
    messages=[...]
```

در این روش برای مدل فرمت پاسخ را تعیین میکنیم ولی رفتار درست و ترتیب لیبل ها تضمین شده نیست .

:3

```
strict_schema = {
    "type": "object",
    "properties": {
        "reasoning": {"type": "string"},
        "label": {
            "type": "string",
            "enum": ["positive", "negative", "neutral"]
        }
    },
    "required": ["reasoning", "label"],
    "additionalProperties": False
}
```

در این روش اسکیما مد نظر خود را تولید میکنیم و به مدل میدهیم تا مطمئن بشویم دقیقا به آن پایبند است

```
response_format={
    "type": "json_schema",
    "json_schema": {
        "name": "sentiment_response",
        "strict": True,
        "schema": strict_schema
    }
},
```

همچنین بخش تعیین فرمت کمی متفاوت است و به جزئیات بیشتری نیاز دارد که به این شرح است

در ادامه برای هر سه روش پرامپت مخصوص تولید کردیم و با تنظیمات مربوطه به مدل دادیم و خروجی را بررسی کردیم که به شرح زیر است

	review	prompt_only_valid	json_mode_valid	strict_schema_valid	prompt_only_output	json_mode_output	strict_schema_output
0	The battery life is stellar, lasting two full ...	True	True	True	{\n "reasoning": "The review expresses a stro...	{\n "reasoning": "The review expresses a posi...	{"reasoning":"The review expresses a positive ...
1	Customer service was utterly unhelpful and rude.	True	True	True	{\n "reasoning": "The review expresses dissat...	{\n "reasoning": "The review expresses dissat...	{"reasoning":"The review expresses dissatisfac...
2	The camera is decent, but the software has fre...	True	True	True	{\n "reasoning": "The review expresses a mixe...	{\n "reasoning": "The review expresses a mixe...	{"reasoning":"The review expresses a mixed sen...
3	Not worth the ticket, though the visual effect...	True	True	True	{\n "reasoning": "The review expresses disapp...	{\n "reasoning": "The review expresses a nega...	{"reasoning":"The review expresses a negative ...
4	It arrived on time.	True	True	True	{\n "reasoning": "The review expresses a posi...	{\n "reasoning": "The review states that the ...	{"reasoning":"The review states that the item ...

به علت اینکه در حال استفاده از مدل قدرتمند gpt-4o-mini هستیم همچین در کد markdown و علائم اضافه رو پاک کردیم هر سه حالت عملکرد خوبی داشتند ولی در بخش های قبل تر این گزارش توضیح دادیم که هر بخش چه تاثیر و عملکردی دارد و تضمین شده ترین روش برای گرفتن خروجی معتبر و دقیق JSON کدام است .

در آزمایش بعدی تاثیر عوض کردن ترتیب استدلال و لیبل در مقابل لیبل و استدلال رو بررسی کردیم که گفتیم در transformer ها حالت بهتر تولید کردن اول استدلال و بعد لیبل هست تا موقع تولید لیبل مدل به استدلال خود attention کند:

```
tricky_review = "Not worth the ticket, though the visual effects were neat."

schema_reasoning_first = {
    "type": "object",
    "properties": {
        "reasoning": {"type": "string"},
        "label": {"type": "string", "enum": ["positive", "negative", "neutral"]}
    },
    "required": ["reasoning", "label"],
    "additionalProperties": False
}

schema_label_first = {
    "type": "object",
    "properties": {
        "label": {"type": "string", "enum": ["positive", "negative", "neutral"]},
        "reasoning": {"type": "string"}
    },
    "required": ["label", "reasoning"],
    "additionalProperties": False
}
```

رفتار مدل با دو رویکرد بالا به شرح زیر است :

```
Reasoning First:
 {"reasoning":"The statement expresses a negative sentiment overall, as it indicates that the experience (likely a movie or event) was not
  worth the ticket price, despite acknowledging some positive aspects (the visual effects). This suggests disappointment or
 dissatisfaction with the overall value.","label":"negative"}

Label First:
 {"label":"negative","reasoning":"The statement expresses disappointment by stating that the experience was 'not worth the ticket,'
 indicating dissatisfaction, despite acknowledging some positive aspects like the visual effects."}
```

دوباره مدل قدرتمند ما وارد عمل شد و باعث شد در هر دو حالت لیبل درست پیش بینی بشود حتی با وجود اینکه در حالت دوم مدل اول لیبل را حدس زده و مدل چیزی برای توجه کردن به توکن های قبلی خود نداشته است.

بخش بعدی بسیار ارزشمند و مربوط به استفاده از ابزار هاست :

در ابتدا در این بخش دو ابزار تعریف میکنیم و برای هر ابزار یک دیکشنری ساختار یافته یا اسکیما تعریف میکنیم که به مدل تعریف و ویژگی ها و محدودیت های ابزار ها را توضیح میدهد

ابزار ها عبارتند از **ماشین حساب** و **تبدیل کننده ارزش واحد های پول**

و سبک ساخت یک **دیکشنری ساختار یافته** برای تعریف یک ابزار به صورت زیر است

```
tools_definition = [
    {
        "type": "function",
        "function": {
            "name": "get_exchange_rate",
            "description": "Get current exchange rate to Tomans for a currency",
            "parameters": {
                "type": "object",
                "properties": {
                    "currency": {"type": "string", "description": "Currency code e.g.
USD, EUR"}
                },
                "required": ["currency"]
            }
        }
    },
    {
        "type": "function",
        "function": {
            "name": "calculator",
            "description": "Calculate mathematical expression string",
            "parameters": {
                "type": "object",
                "properties": {
                    "expression": {"type": "string", "description": "e.g. '50 *
60000'"}
                },
                "required": ["expression"]
            }
        }
    }
]

available_tools = {
    "get_exchange_rate": get_exchange_rate,
    "calculator": calculator
}
```

باید نوع ، توضیحات فانکشن (شامل نام ، تعریف معین و کوتاه و یک خطی ، پارامتر ها ، پارامتر های اجباری و... تعیین شود) سپس انواع ابزار های موجود را در این غالب ذخیره کرده و هنگام پرامپت زدن به مدل داد تا در صورت نیاز تصمیم به استفاده بگیرد .

```
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=messages,
    tools=tools_definition,
    tool_choice="auto",
    temperature=0
)
```

سپس در این غالب به مدل **دیکشنری تعریف ابزار ها** را میدهیم و اجازه انتخاب بین انها را فعال میکنیم و پرامپت میدهیم

سپس در یک حلقه با یک حداکثر تکرار یا **گاردریل** مشخص پرامپت میدهیم و تا زمانی که پرامپت نهایی و بدون **call** کردن ابزار را تولید نکند به پرامپت زدن ادامه میدهیم و هر بار **پیام اولیه کاربر** ، **ابزار ها** و **id خروجی ابزار ها** را اضافه میکنیم و وقتی مدل دوباره پرامپت را میبیند متوجه مراحل قبلی اجرای خود می شود و هنگام تولید پرامپت مراحل قبلی را از یاد نمی برد:

```
Step 1: Called 'get_exchange_rate' with {'currency': 'USD'} -> Output: 60000.0
Step 2: Called 'calculator' with {'expression': '50 * 60000'} -> Output: 3000000.0
Step 3: Called 'calculator' with {'expression': '3000000 * 0.10'} -> Output: 300000.0
Final Answer (Reached in step 4):
50 dollars is equivalent to 3,000,000 tomans. Ten percent of that amount is 300,000 tomans.
```

اینجا step های مختلف مدل برای تولید پاسخ را مشاهده میکنید و همان طور که میبینید مدل در آخرین تکرار جمع بندی و درک خود از خروجی ابزار ها و پرامپت های مرحله قبل را تولید کرده

نتیجه گیری این بخش : مدل قادر به کاری جز **تولید متن** نیست ، میتوانیم ابزار هایی تولید کنیم و در اختیار مدل بگذاریم و در یک **حلقه** با **محدودیت** استفاده از **ابزار ها** و **تکرار** (برای سوزانده نشدن همه توکن ها در حلقه بی نهایت) به مدل اجازه دهیم از ابزار استفاده کند و پاسخ خود را به کمک آنها ارائه دهد .

علت این کار این است که مدل ها ممکن است دچار توهم بشوند و در یک حلقه بی نهایت دائم درخواست استفاده از ابزار بکنند برای همین برای آنها محدودیت تعریف میکنیم تا منابع در اختیار پروژه را تمام نکنند .

پاسخ به دو سوال نهایی :

1. **تفاوت فرم معتبر با جواب درست:** اعتبار سنجی فرمت صرفا ، نوع داده قواعد ساختاری و نام فیلد ها را تضمین میکند و مانع کرش کردن ابزار ها میشود اما هیچ تضمینی برای صحت معنا ندارد پس ممکن است پاسخ غلط ولی فرمت درست باشد .
2. **چرا کد باید خودش مجوز ها و دسترسی های ابزار را کنترل کند :** مدل های زبانی به دلیل ماهیت احتمالی در برابر حملات تزریق پرامپت و تولید توکن های پیش بینی نشده آسیب پذیرند و درکی از امنیت سیستم ندارند حالا اگر اجرای ابزار ها بدون اعتبار سنجی ورودی و محدود سازی سطح دسترسی و فیلتر کردن توابع مجاز به مدل واگذار شود با ریسک هایی مثل **اجرای کد های مخرب** ، یا **دستکاری داده های حساس** روبرو خواهیم شد پس همواره این امنیت باید در لایه برنامه اعمال بشود (فرض کنید یک نفر به جای یک ماشین حساب ایمن از ()eval در پایتون استفاده کند ، در این صورت مدل هر کدی که بخواهد میتواند اجرا کند و کاربر به سادگی با فریب مدل میتواند کد های مخرب تولید کند)